T5 - Pandas and NumPy: load, clean, summarise, write

Covers NumPy arrays, pandas CSV loading, missing values, dates, groupby and writing results. Earns the marks of 2024 Task 1a (9 of 13) and the pandas/NumPy half of "Practice Exam Task 3" (statistics + summary file); it also feeds every chart in T6.

1. What the exam asks

2024 final lab exam, Python Task 1 [13 marks] - file `sales_data.csv` (columns `Date, Product, Quantity, Price, Total`):

> **Task 1a: Data Loading and Preprocessing** - Load the CSV file into a Pandas DataFrame. [2 mark] - Ensure there are no missing values. If any are found, fill them with appropriate values. [3 mark] - Convert the Date column to a datetime object. [2 mark] - Ensure the Total column correctly represents the product of Quantity and Price. If any discrepancies are found, correct them. [2 mark]

Practice Exam Task 3 (replaces PyTorch) - Task 3 "Summary Statistics":

> After saving the data, use pandas and Numpy to: read the CSV file into a DataFrame; print the total number of records; print the average, minimum, and maximum heights; save summary results to a text file called `height_summary.txt`.

(The heights *logger* - appending rows, `q` to quit - is in the T3 document.)

Where the course taught this: `WEEK3/Class4/Data_Manipulation.ipynb` (the shopping.csv walk-through), the slides "More Data Manipulation - date format and missing data" (slides 3-9: `parse_dates`, `dayfirst`, `isnull().sum()`, `fillna({...})`), "Pandas-reading and writing", the NumPy masking slides, and `WEEK3/Lab04` (`Lab4_solutions.pdf` Tasks 1-13, `Lab 4_2` indexing exercises). The 2022 and 2023 papers had no pandas; 2024 is the pattern to expect.

What the 2024 data really contains (checked on the real file): 151 rows; dates are text like `01/01/2023` (dd/mm/yyyy); years 2022 (29 rows), 2023 (100), 2024 (22); 6 blank `Product`, 1 blank `Price`, 3 blank `Total`; no blank `Date` or `Quantity`; every non-blank Total already equals Quantity x Price. **CORRECTION (added later):** the file also holds **6 exact duplicate rows** (the 04/04/2022 to 10/04/2022 sales appear twice; rows 144-150 repeat rows 122-128). The 2024 Task 1a does not ask you to remove them, but Task 2a says "ensure no duplicate entries" - see section 3.12.

1.1 The instructor's checklist - Pandas bullets covered here

> **Pandas** (practical-assessment checklist from the course instructor) - loading CSV files -> template B - checking the shape of a DataFrame -> **3.11** - identifying and removing duplicates -> **3.12** (and template D) - merging DataFrames using a common column -> **3.13** - selecting and filtering data -> template C and **3.14** - saving a DataFrame to a CSV file -> template H and **3.15**

The instructor says: do not memorise one complete solution; practise the *individual operations* and know what each line does. Sections 3.11-3.15 are therefore small, independent recipes: each starts from a fresh DataFrame.

Column	Blank means	Fill with	Reason
Price	price not recorded	`Total / Quantity` first, else `0.0`	the data lets us recover it exactly (row 70: 50/5 = 10)
Total	total not recorded	`Quantity * Price`	it is a derived column - step 4 does this for everyone
Quantity	(none blank here)	`0`	defensive default in case the exam file differs
Date	(none blank here)	do NOT invent; `errors='coerce'` then `dropna(subset=['Date'])`	a made-up date corrupts the monthly charts

TIP: the course solution (`ProgSD-Python-Solution.py`) fills Price with `0.0` and then recomputes Total. That is accepted, but it turns the row with the blank Price (Chocolate, 12/03/2023, Total 50) into Total = 0 and under-reports March 2023 sales by 50. Recovering the Price first is strictly better and costs three lines.

H. Writing results to files (Practice Exam Task 3, step 3)

```
heights = pd.DataFrame({
    'Name': ['Amy', 'Ben', 'Ali', 'Yin', 'Mohammed', 'Gus'],
    'Height': [165, 172, 166, 180, 170, 182]})
heights.to_csv('heights.csv', index=False)     # index=False: no 0,1,2 column

h = pd.read_csv('heights.csv')                 # read it back
count = len(h)                                 # number of records (6)
avg = h['Height'].mean()                       # or np.mean(h['Height'])
low, high = h['Height'].min(), h['Height'].max()
print('Records:', count)
print(f'Average: {avg:.1f}  Min: {low}  Max: {high}')   # 172.5 165 182

with open('height_summary.txt', 'w') as f:     # text report, closed for us
    f.write(f'Total records: {count}\n')
    f.write(f'Average height: {avg:.2f} cm\n')
    f.write(f'Minimum height: {low} cm\n')
    f.write(f'Maximum height: {high} cm\n')

df.to_csv('cleaned_sales.csv', index=False)    # save the cleaned sales too
df['Date'].dt.strftime('%Y-%m-%d')             # datetime -> 'YYYY-MM-DD' text
                                               # (what SQLite wants, see T4)
```

3.11 Check the shape (do it before AND after every step)

Analogy: counting the books on the shelf before and after you tidy - if the number is not what you expect, a step did something you did not intend.

```
df = pd.read_csv('sales_data.csv')
print(df.shape)                      # (151, 5)  = (rows, columns)
print(len(df))                       # 151       = rows only (len = rows)
rows, cols = df.shape                # unpack the pair into two names
print(f'{rows} rows x {cols} columns')
print(df.size)                       # 755 = rows x columns = all cells
df.info()                            # columns, non-null counts, dtypes

n_before = len(df)
clean = df.drop_duplicates()         # any step that can change row count
print('removed', n_before - len(clean), 'rows')   # removed 6 rows
print(df.shape, '->', clean.shape)   # (151, 5) -> (145, 5)
```

- `df.shape[0]` = rows, `df.shape[1]` = columns. `shape` has NO brackets (it is a value, not a function). `df.info()` prints by itself.
- `df.info()` "Non-Null Count" below the row count = blanks in that column.

3.12 Identify and remove duplicates

Analogy: a class register where one student signed twice. First *find* the double signatures, *look* at them, then rub one out.

```
df = pd.read_csv('sales_data.csv')
print(df.duplicated().sum())            # 6   = how many rows repeat an
                                        #       earlier row (True = repeat)
print(df[df.duplicated(keep=False)].sort_values('Date'))   # SEE both copies
print(df[df.duplicated()].index.tolist())   # row numbers of repeats

before = df.shape                       # (151, 5)
dedup = df.drop_duplicates()            # keeps the FIRST copy of each row
dedup = dedup.reset_index(drop=True)    # renumber 0,1,2... (drop old numbers)
print(before, '->', dedup.shape)        # (151, 5) -> (145, 5)

# judge "duplicate" by some columns only:
same_day_product = df.duplicated(subset=['Date', 'Product']).sum()
nodup = df.drop_duplicates(subset=['Date', 'Product'], keep='last')
print(same_day_product, nodup.shape)    # 6 (145, 5)
```

Call	What it does
`df.duplicated()`	True/False per row: True if an EARLIER row is identical
`.duplicated().sum()`	how many repeat rows (True counts as 1)
`df[df.duplicated(keep=False)]`	show ALL copies (original + repeats) so you can inspect them
`drop_duplicates()`	delete repeats, keep the first (`keep='first'`, default)
`drop_duplicates(keep='last')`	keep the last copy instead
`drop_duplicates(keep=False)`	delete EVERY copy, including the original (137 rows left!)
`subset=['A', 'B']`	only compare these columns when deciding "same"
`reset_index(drop=True)`	tidy row numbers after rows vanish (`drop=True`: do not keep the old numbers as a column)

TIP: do not use `inplace=True`; write `df = df.drop_duplicates()`. **NOTE:** the exam text "ensure no duplicate entries are added" (Task 2a) also needs de-duplication in the database (T4); in pandas do it first.

3.13 Merge two DataFrames on a common column

Analogy: two guest lists, one with names, one with meal choices; the ticket number on both lists is the *common column*. Merging = stapling each name to the meal with the same ticket number.

```
students = pd.DataFrame({'ID': [1, 2, 3, 4],
                         'Name': ['Amy', 'Ben', 'Cat', 'Dev']})
scores = pd.DataFrame({'ID': [1, 2, 5],
                       'Score': [70, 80, 60]})
students.to_csv('students.csv', index=False)    # pretend they are two files
scores.to_csv('scores.csv', index=False)

a = pd.read_csv('students.csv')
b = pd.read_csv('scores.csv')
print(a.shape, b.shape)                         # (4, 2) (3, 2)

both = pd.merge(a, b, on='ID')                  # same as a.merge(b, on='ID')
print(both)                                     # only IDs 1 and 2 survive
print(a.shape, b.shape, '->', both.shape)       # (4, 2) (3, 2) -> (2, 3)

left = a.merge(b, on='ID', how='left')          # keep ALL students
print(left)                                     # Cat, Dev get Score NaN
print(left['Score'].isna().sum())               # 2 students with no score

full = a.merge(b, on='ID', how='outer', indicator=True)
print(full)                        # _merge: both/left_only/right_only
```

How the four joins differ (`a` = left table, `b` = right table):

`how=`	Rows kept	Rows in example
`'inner'` (default)	ID found in BOTH tables	2 (IDs 1, 2)
`'left'`	every row of `a`; blanks (NaN) where `b` has no match	4

how=	Rows kept	Rows in example
'right'	every row of b; blanks where a has no match	3
'outer'	every row of both; blanks wherever one side is missing	5

indicator=True adds a _merge column that says where each row came from - the quickest way to *see* unmatched rows.

Different column names, repeated names, repeated keys:

```
scores2 = b.rename(columns={'ID': 'StudentID'})
m1 = a.merge(scores2, left_on='ID', right_on='StudentID')   # keys named
print(m1.columns.tolist())      # ['ID', 'Name', 'StudentID', 'Score'] - both
                                # both keys stay; drop one if unwanted

t1 = pd.DataFrame({'ID': [1, 2], 'Mark': [1, 2]})
t2 = pd.DataFrame({'ID': [1, 2], 'Mark': [9, 8]})
print(t1.merge(t2, on='ID', suffixes=('_test', '_exam')))  # Mark_test..
print(t1.merge(t2, on='ID').columns.tolist())   # default: Mark_x, Mark_y

# DUPLICATE KEYS multiply rows: 3 rows x 2 matching rows = 6
x = pd.DataFrame({'k': [1, 1, 1], 'a': [1, 2, 3]})
y = pd.DataFrame({'k': [1, 1], 'b': [7, 8]})
print(x.merge(y, on='k').shape)                 # (6, 3)  <- rows blew up
print(a['ID'].is_unique, b['ID'].is_unique)     # True True: safe 1-to-1 merge
```

Real-data version (add a supplier to every sale; keep all 151 sales):

```
sales = pd.read_csv('sales_data.csv')
info = pd.DataFrame({'Product': ['Water', 'Sweets'],
                     'Supplier': ['AquaCo', 'SweetCo']})
joined = sales.merge(info, on='Product', how='left')
print(sales.shape, '->', joined.shape)          # (151, 5) -> (151, 6)
print(joined['Supplier'].isna().sum())          # 90 sales of other products
```

- **Rule of thumb:** after a *left* merge the row count must equal the left table's (if not, the right table has duplicate keys). An *inner* merge can only lose rows. Always print shape before and after.
- Merge keys must have the SAME type (int with int); df['ID'].astype(int) to fix.

3.14 Selecting and filtering - more tools

```
df = pd.read_csv('sales_data.csv')
one = df['Total']                               # one column  -> Series
many = df[['Product', 'Total']]                 # list of names -> DataFrame
row5 = df.iloc[5]                               # row by POSITION
sub = df.loc[df['Quantity'] > 8, ['Product', 'Total']]   # rows AND columns
print(sub.head(2))

# filters (each condition in its own brackets; & = and, | = or, ~ = not)
a = df[(df['Product'] == 'Water') | (df['Quantity'] > 8)]      # 61 rows
b = df[~df['Product'].isin(['Water'])]                         # not Water
c = df[df['Quantity'].between(5, 7)]            # 5 <= Quantity <= 7 inclusive
d = df[df['Product'].str.contains('Orange', na=False)]         # text search
e = df[df['Product'].str.startswith('S', na=False)]            # starts with S
f = df[df['Product'].notna()]                   # drop blank products
print(len(a), len(b), len(c), len(d), len(e), len(f))   # 61 119 58 31 29 145

# query(): the same filter written as a string (read it like English)
lim = 6
g = df.query("Quantity > 5 and Product == 'Water'")   # 0 rows
h = df.query("Product in ['Water', 'Sweets']")             # 61 rows
k = df.query("Quantity > @lim")                 # @ = use a Python variable
print(len(g), len(h), len(k))                   # 0 61 90

# sorting and "top n"
print(df.sort_values('Total', ascending=False).head(3))
print(df.sort_values(['Product', 'Quantity'],
                     ascending=[True, False]).head(2))
```

```
print(df.nlargest(2, 'Total')[['Product', 'Total']])   # 2 biggest sales
print(df.loc[df['Product'] == 'Water', 'Price'].mean())   # 7.5 (one number)
```

Tool	Use for
`df['A']`, `df[['A','B']]`	pick columns (single [] = Series, double [] = DataFrame)
`df.loc[rows, cols]`	by label / by condition; end of a slice INCLUDED
`df.iloc[r, c]`	by position; end EXCLUDED
`df[cond]`	filter rows; combine with `&`, `|`, `~` and brackets
`isin([..])`, `between(a, b)`	"is one of" / "is in range (inclusive)"
`str.contains('x', na=False)`	text search; `na=False` stops a crash on blanks (older pandas)
`query("A > 5 and B == 'x'")`	filter as a string; `@var` for variables

3.15 Save to CSV - and check by reading it back

```
df = pd.read_csv('sales_data.csv')
big = df[df['Quantity'] > 8]                        # a filtered subset

big.to_csv('big_sales.csv', index=False)            # whole subset, no index
big[['Date', 'Product']].to_csv('big_two_cols.csv', index=False)  # 2 columns
df.to_csv('no_header.csv', index=False, header=False)   # data rows only
df.to_csv('semicolon.csv', index=False, sep=';')   # other separator

back = pd.read_csv('big_sales.csv')                 # read it back to verify
print(back.shape, big.shape)                        # (29, 5) (29, 5)
print(back.columns.tolist())                        # same column names
print(back.head(2))
print(pd.read_csv('big_two_cols.csv').columns.tolist())   # 2 names
```

- `index=False` - without it pandas writes the row numbers as an extra first column; reading the file back then shows a column `Unnamed: 0`.
- `header=False` writes no column names; read it back with `pd.read_csv(f, header=None)`, otherwise the first data row becomes the header and you lose a row (150 instead of 151).
- Saving does NOT change `df`; saving a *filtered* DataFrame saves only those rows. Dates written from a datetime column come back as text - run `to_datetime` again after reading.
- Verify: `shape` of the re-read file equals `shape` of what you saved.

Mistake	Error you see	Fix
`ydf = df[df['Quantity'] > 8]; ydf['X'] = 1`	no warning in pandas 3, and `df` is unchanged (a filter makes a copy)	assign on `df` itself, or `.copy()` deliberately
`df.loc[0:2]` vs `df.iloc[0:2]`	loc gives 3 rows (end included), iloc gives 2	remember: loc = labels, inclusive
`value_counts()` column names	pandas 3 names the counts `count`; older versions name them differently	index by position, not by name

NumPy traps

- `a[(a > 1) and (a < 3)]` -> `ValueError: truth value of an array ...` - use `&` with brackets.
- `[1, 2, 3] > 2` -> `TypeError` (a *list* is not an array; wrap in `np.array`).
- `b = a[:2]; b[0] = 99` **also changes** `a` (a slice is a view). Use `a[:2].copy()` when you need independence.
- `a + np.array([1, 2])` with different lengths -> `ValueError: operands could not be broadcast`. Shapes must match (or one must be a scalar/row).
- `a.mean()` returns `np.float64`; `print(a.mean())` is fine but a *list* of them shows `np.float64(172.5)`. Use an f-string `f'{x:.1f}'` or `float(x)`.
- `np.arange(1, 10)` stops BEFORE 10; `np.linspace(0, 1, 5)` includes both ends.

Other

- `FileNotFoundError` on `read_csv`: the CSV is not next to the `.py` file (check the working directory) or the name differs (`sales_data.csv`).
- `df.info()` already prints; writing `print(df.info())` adds a stray `None`.
- Pandas 3 shows text columns as `str`; older versions say `object`. Both mean "text".
- Never type `df.isna.sum()` - `isna` needs brackets: `isna().sum()`.

5.1 Traps for sections 3.11-3.15

Mistake	What you see	Fix
`df.shape()`	`TypeError: 'tuple' object is not callable`	`shape` has no brackets; `info()` does
`print(df.info())`	table, then a stray `None`	just `df.info()`
`df.drop_duplicates()` without assigning	nothing changes (`df` still 151 rows)	`df = df.drop_duplicates()`
`drop_duplicates(keep=False)`	removes originals too (137 rows left, not 145)	default `keep='first'`
duplicates found with `subset=['Product']`	145 "duplicates": same product, different sale	choose a subset that identifies a record, or none
row numbers 0,1,2,..,5,7 after dropping rows	gaps in the index	`reset_index(drop=True)`
merge with wrong key name	`KeyError: 'id'` (case matters)	check `list(a.columns)`; use `left_on=` / `right_on=`
merge int column with text column	`ValueError: You are trying to merge on int64 and str columns`	`astype` one side so both match
merge result has far MORE rows than the left table	duplicate keys multiplied rows (3 x 2 = 6)	`df['k'].is_unique`; `drop_duplicates(subset='k')` first, or `validate='one_to_one'` (raises `MergeError` if not)
merge result has far FEWER rows	default is `how='inner'`: unmatched rows vanish	`how='left'` or `how='outer', indicator=True` to see them
both tables have a `Mark` column	result has `Mark_x`, `Mark_y`	`suffixes=('_test', '_exam')` or rename first

Mistake	What you see	Fix
`df[df['Product'].str.contains('x')]`	older pandas: `ValueError: Cannot mask with non-boolean array containing NA`	`na=False`
`df.query("Quantity > lim")`	`UndefinedVariableError`	`@lim`
`df.loc[df['Quantity'] > 8, 'Total']` vs `df[df['Quantity'] > 8]['Total']`	both read; only the first is safe for assigning	use `.loc[rows, col]` for assignment
`to_csv` without `index=False`	extra `Unnamed: 0` column on read-back	`index=False`
re-reading a saved file with `header=False`	first row used as header, 1 row lost	`header=None` when reading

6. Anticipated exam questions

Q1 (2) Load `sales_data.csv` and show the first rows and the shape.

```
df = pd.read_csv('sales_data.csv')
print(df.head())
print(df.shape)                        # (151, 5)
```

Q2 (3) Ensure there are no missing values; fill with appropriate values and explain your choices. Use template G. Say: text -> `'Unknown'`; Price -> `Total / Quantity` where possible, else 0; Total is recomputed from Quantity x Price; do not invent dates. Prove it with `df.isna().sum()`.

Q3 (2) Convert Date to datetime and add Year and Month columns.

```
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
```

Q4 (2) Make Total equal Quantity x Price; report how many rows were wrong.

```
expected = df['Quantity'] * df['Price']            # what Total should be
wrong = ~np.isclose(df['Total'], expected)         # True = mismatch or blank
print(wrong.sum(), 'rows had a wrong Total')       # raw file: 4
df['Total'] = expected
```

Q5 (2) Which product sold the most units, and how many?

```
per_prod = df.groupby('Product')['Quantity'].sum()
print(per_prod.idxmax(), per_prod.max())       # Sweets 261
```

Q6 (3) Print the total sales for 2023 and the average sale per product for 2023.

```
y23 = df[df['Date'].dt.year == 2023]
print(y23['Total'].sum())
print(y23.groupby('Product')['Total'].mean().round(2))
```

Q7 (2) Remove duplicate rows and rows with no product, then save the result to `cleaned_sales.csv`.

```
out = df.drop_duplicates().dropna(subset=['Product'])
out.to_csv('cleaned_sales.csv', index=False)
```

Q8 (3) (Practice Task 3) Read `heights.csv`; print the number of records and the average, minimum and maximum height; save them to `height_summary.txt`. Template H.

Q9 (3) (NumPy) Given `a = np.array([12, 7, 30, 18, 25])`, print the values above 15, replace values below 10 with 0, and print the mean.

```
a = np.array([12, 7, 30, 18, 25])
print(a[a > 15])                       # [30 18 25]
a[a < 10] = 0
print(a, a.mean())                     # [12  0 30 18 25] 17.0
```

6.1 Extra questions on the instructor's checklist

Q10 (2) Load the file, print how many rows and columns it has, and how many rows have a missing value.

```
df = pd.read_csv('sales_data.csv')
rows, cols = df.shape
print(rows, 'rows,', cols, 'columns')              # 151 rows, 5 columns
print(df.isna().any(axis=1).sum(), 'rows with a blank')   # 10
```

Q11 (3) Find the duplicate rows, display them, remove them, and show the shape before and after.

```
df = pd.read_csv('sales_data.csv')
print(df.duplicated().sum())                       # 6
print(df[df.duplicated(keep=False)].sort_values('Date'))
```

```
print('before:', df.shape)
df = df.drop_duplicates().reset_index(drop=True)
print('after:', df.shape)                          # (145, 5)
```

Q12 (4) `students.csv` (ID, Name) and `scores.csv` (ID, Score): merge them on ID so that every student appears, even without a score. Report how many have no score.

```
a = pd.read_csv('students.csv')
b = pd.read_csv('scores.csv')
m = a.merge(b, on='ID', how='left')
print(m.shape, a.shape)                            # same row count as a
print(m['Score'].isna().sum(), 'students without a score')
```

Q13 (2) What is the difference between an inner and a left merge? Inner keeps only keys present in both tables; left keeps every row of the left table and fills the right-hand columns with NaN where there is no match.

Q14 (3) Select the Water and Sweets sales with Quantity of at least 5, keep only Date, Product and Total, sort by Total (largest first) and save to `water_sweets.csv`.

```
df = pd.read_csv('sales_data.csv')
sel = df[df['Product'].isin(['Water', 'Sweets']) & (df['Quantity'] >= 5)]
sel = sel[['Date', 'Product', 'Total']].sort_values('Total', ascending=False)
sel.to_csv('water_sweets.csv', index=False)
print(pd.read_csv('water_sweets.csv').shape)       # (29, 3)
```

Q15 (2) Why does `df.merge(other, on='k')` give more rows than `df` has? `other` contains the same key more than once, so each matching row is repeated for every match (rows multiply). Check `other['k'].is_unique`.

7. Quick-reference box (hand-copy this)

```
LOAD     df = pd.read_csv('f.csv')   df.head() .shape .info() .describe()
COLUMNS  df['A']   df[['A','B']]   df.loc[r,'A']   df.iloc[r,c]
FILTER   df[(df['A']>5) & (df['B']=='x')]     (& | ~ , brackets!)
BLANKS   df.isna().sum()
FILL     df = df.fillna({'Prod':'Unknown','Price':0.0})   [text->text]
         df['P'] = df['P'].fillna(df['P'].mean())
         NOT  df['P'].fillna(..., inplace=True)   (does nothing in pandas 3)
DROP     df.dropna(subset=['A'])   df.drop_duplicates()   .astype(int)
DATES    df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)  # dd/mm/yyyy
         .dt.year  .dt.month  .dt.to_period('M')
         df[df['Date'].dt.year == 2023]
NEW COL  df['Total'] = df['Quantity'] * df['Price']
GROUP    df.groupby('Prod')['Qty'].sum()   .mean()  .size()  .sort_values()
COUNT    df['Prod'].value_counts()   .idxmax()   .max()   .apply(lambda x: ..)
WRITE    df.to_csv('out.csv', index=False)   open('f.txt','w').write(..)
NUMPY    np.array([..])  .shape  a[1:4]  m[r,c]  m[:,0]  a[a>5]  a[[0,2]]
         a.sum()/mean()/min()/max()   m.sum(axis=0) per column, axis=1 per row
TRAPS    parse_dates needs dayfirst=True | use & not and
         text column -> 'Unknown', number column -> number
--- instructor checklist additions ---
SHAPE    df.shape (no brackets) -> (rows, cols)   len(df)   df.info()
DUPES    df.duplicated().sum()   df[df.duplicated(keep=False)] (see them)
         df = df.drop_duplicates(subset=[..], keep='first'|'last'|False)
         .reset_index(drop=True)   print shape before -> after
MERGE    a.merge(b, on='ID', how='inner'|'left'|'right'|'outer')
         left_on='x', right_on='y'  suffixes=('_a','_b')  indicator=True
         duplicate keys -> rows multiply; left merge keeps len(a)
FILTER   df[~df['A'].isin([..])]  .between(a,b)  .str.contains('x', na=False)
         df.query("A > 5 and B == 'x'")   @var   df.nlargest(3,'Total')
SAVE     df.to_csv('f.csv', index=False)  [['A','B']]  header=False
         verify: pd.read_csv('f.csv').shape == df.shape
```

T6 - Matplotlib: bar, line, histogram, scatter, pie, subplots

Covers every chart the exam can ask for and how to build it from a pandas groupby or a plain list. Earns 2024 Task 1b (4 marks), the loss plot of 2024 Task 3d (3 marks), and the "Height Distribution" histogram of Practice Exam Task 3.

1. What the exam asks

2024 final lab exam, Task 1b: Data Visualisation (on `sales_data.csv`)

> **Product Sales Distribution:** Create a bar chart showing the total quantity sold for each product. [2 mark]
>
> **Sales Over Time:** Create a line plot to show the trend of total sales over the months of 2023. The x-axis should represent the months, and the y-axis should represent the total sales. [2 mark]

2024 Task 3d: Loss Visualisation [3 marks]

> Plot the training loss over epochs using matplotlib. Your plot should display the training loss decreasing over time, indicating that the model is learning.

Practice Exam Task 3, Task 4: Visualisation (replaces PyTorch)

> Using matplotlib, create a simple histogram showing the distribution of heights. Title: "Height Distribution". X-axis label: Height (cm). Y-axis label: Frequency.

Where the course taught this: `WEEK3/Class_Lab_5` (`Matplotlib_study_notes.md`, `Lab5 Exercise – solutions.pdf` Tasks 1-19, `Lab5.docx`), and the last two cells of `WEEK3/Class4/Data_Manipulation.ipynb` (bar per product and monthly line chart from a groupby). The marker looks for: **the right chart type, a title, labelled axes, and the right numbers on it.** The 2022 and 2023 Python papers had no charts.

1.1 The instructor's checklist - Matplotlib bullets covered here

> **Matplotlib** (practical-assessment checklist from the course instructor) - creating simple plots such as scatter plots, line plots, bar charts and box plots - adding appropriate titles and axis labels
>
> -> scatter, line, bar: independent templates in **3.11**; box plots: **3.12**; choosing the plot: **3.13**. Titles and both axis labels are in EVERY template.

The instructor's advice: do not memorise one complete solution; practise the *individual operations*. Sections 3.11-3.13 are small independent recipes.

CORRECTION to the "expected numbers" in template K: the real `sales_data.csv` contains 6 exact duplicate rows (04/04/2022 to 10/04/2022 appear twice). The official solution does not remove them in Task 1, and the numbers quoted in template K are with the duplicates left in. If you call `df.drop_duplicates()` first, the quantities become Biscuits 196, Chocolate 120, Orange Juice 232, Sweets 252, Unknown 33, Water 90 (the 2023 monthly chart is unchanged: the duplicates are all in 2022).

```
plt.savefig('chart.png')              # to a file (type from extension)
plt.savefig('chart.png', dpi=300, bbox_inches='tight')   # sharper, trimmed
plt.show()                                    # AFTER savefig, never before
```

- The exam says "plot", so `plt.show()` is what is marked. Add a `savefig` line only if the question says "save".
- With several figures, `plt.savefig` saves the *current* (latest) one; use `fig.savefig(...)` on a named figure (`fig = plt.figure()`).

3.11 Scatter, line and bar: three independent minimal templates

Each one stands alone (own data, own captions). Swap in your own lists or DataFrame columns.

Scatter - is there a relationship between two numbers?

```
hours = [1, 2, 3, 4, 5, 6, 7, 8]           # x: hours studied
marks = [35, 42, 50, 55, 62, 70, 74, 85]   # y: mark achieved (same length!)
plt.figure()
plt.scatter(hours, marks, color='purple', s=60, alpha=0.7, marker='o')
plt.title('Study Time vs Exam Mark')
plt.xlabel('Hours studied')
plt.ylabel('Exam mark')
plt.show()
```

Part	What it does
`scatter(x, y)`	one dot per (x, y) pair; no line joins them
`color=`	dot colour (`'red'`, `'purple'`, `'#1f77b4'`)
`s=60`	dot SIZE (area); bigger number = bigger dot
`alpha=0.7`	transparency, 0 (invisible) to 1 (solid); helps when dots overlap
`marker='o'`	dot shape (`'s'` square, `'^'` triangle, `'x'` cross)
`label=`	name for `plt.legend()` when you draw several scatters

Line - how does a value change over time/order?

```
months = [1, 2, 3, 4, 5, 6]
sales = [210, 180, 225, 220, 205, 260]
plt.figure()
plt.plot(months, sales, color='darkgreen', marker='o',
         linestyle='--', linewidth=2, label='Sales')
plt.title('Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Units sold')
plt.legend()
plt.grid(True)
plt.show()
```

Part	What it does
`plot(x, y)`	joins the points IN THE ORDER GIVEN (sort by time first)
`marker='o'`	a dot at every data point
`linestyle='--'`	`'-'` solid, `'--'` dashed, `':'` dotted
`linewidth=2`	line thickness
`label='Sales'` + `plt.legend()`	key box; no `legend()` call = no key
`plt.grid(True)`	light grid to read values

Bar - compare categories

```
products = ['Water', 'Sweets', 'Biscuits']
totals = [720, 3915, 4060]
plt.figure()
plt.bar(products, totals, color='steelblue', width=0.6, edgecolor='black')
plt.title('Total Sales per Product')
plt.xlabel('Product')
```

```
plt.ylabel('Total sales')
plt.show()
# horizontal: plt.barh(products, totals) – then xlabel is the number axis
```

Part	What it does
`bar(names, heights)`	one bar per category; names on x, heights on y
`color=`	one colour, or a list with one colour per bar
`width=0.6`	bar width (default 0.8; smaller = bigger gaps)
`edgecolor='black'`	outline, nice on printed paper
`plt.barh(names, widths)`	horizontal bars: good for long names
`yerr=`	error bars (one value per bar)

3.12 Box plots

Analogy: a box plot is a **shrink-wrapped summary of a pile of numbers**. Line up all the values from smallest to largest, cut the row into four equal groups, and draw the middle two groups as a box. The line across the box is the middle person in the row (the median).

How to read one:

Part	Meaning
line inside the box	**median** (50% of values lie below it)
bottom / top of box	**Q1** (25th percentile) and **Q3** (75th percentile)
box height	**IQR** = Q3 - Q1, the spread of the middle 50%
whiskers	reach to the furthest value within 1.5 x IQR of the box
separate dots beyond whiskers	**outliers** (unusually high/low values)
tall box / long whiskers	values very spread out; short box = consistent

Rule of thumb: compare boxes by **median** (height of the line), **spread** (box height) and **outliers** (dots).

```
class_a = [55, 60, 62, 65, 67, 70, 72, 75, 78, 95]
class_b = [40, 48, 55, 58, 60, 62, 65, 70, 80, 85]
class_c = [66, 68, 69, 70, 71, 72, 72, 73, 74, 75]

plt.figure(figsize=(6, 4))
plt.boxplot([class_a, class_b, class_c])      # a LIST of lists: one box each
plt.xticks([1, 2, 3], ['Class A', 'Class B', 'Class C'])   # name the boxes
plt.title('Exam Marks by Class')
plt.xlabel('Class')
plt.ylabel('Mark')
plt.tight_layout()
plt.show()
# reading it: Class A has the highest median and ONE outlier (95);
# Class B is the most spread out; Class C is very consistent (short box).
```

Naming the boxes - version warning (tested on Matplotlib 3.11):

```
plt.boxplot(data, tick_labels=['A','B'])   # Matplotlib 3.9 and newer: works
plt.boxplot(data, labels=['A','B'])        # OLD name. In 3.11 this gives
   TypeError: boxplot() got an unexpected keyword argument 'labels'
plt.boxplot(data, vert=False)              # old horizontal switch
   warns it is deprecated; use orientation='horizontal'
```

The safe way that works in EVERY version: leave the keyword out and call `plt.xticks([1, 2, 3], [names])` after `plt.boxplot(..)`, as above. Boxes are numbered 1, 2, 3, ... (not 0).

Box plot straight from a DataFrame (the long format of Lab5.docx Task 12: one column of groups, one of values)

```
marks = pd.DataFrame({'Class': ['A'] * 10 + ['B'] * 10 + ['C'] * 10,
                      'Mark': class_a + class_b + class_c})
marks.boxplot(column='Mark', by='Class', grid=False, figsize=(6, 4))
plt.suptitle('')                # remove pandas' automatic extra title
```

```
plt.title('Exam Marks by Class')
plt.xlabel('Class')
plt.ylabel('Mark')
plt.show()

# one box per COLUMN of a DataFrame:
wide = pd.DataFrame({'Class A': class_a, 'Class B': class_b,
                     'Class C': class_c})
wide.plot(kind='box', title='Exam Marks by Class')
plt.ylabel('Mark')
plt.show()

df = pd.read_csv('sales_data.csv')            # a single column: one box
df['Total'].plot(kind='box', title='Total per sale')
plt.ylabel('Total')
plt.show()
```

- `column='Mark'` = the numbers to summarise; `by='Class'` = make one box per distinct value of this column.
- Boxes for the sales file look flat when split by Product, because every product has one fixed price: that is the data, not a mistake.
- Quartiles of the sales `Total` column: 50 / 100 / 135 (Q1 / median / Q3).

3.13 Which plot for which question?

Question	Plot	Call
Are two numbers related? (hours vs mark)	**scatter**	`plt.scatter(x, y)`
How does it change over time / order? (monthly sales)	**line**	`plt.plot(x, y)`
How do categories compare? (total per product)	**bar**	`plt.bar(names, heights)`
How are the values spread? (heights of 30 students)	**histogram**	`plt.hist(values, bins=..)`
How do SEVERAL groups differ in spread, median and outliers?	**box plot**	`plt.boxplot([g1, g2, g3])`
What share of a whole does each part take?	pie	`plt.pie(parts, labels=..)`

Clue words: "relationship / correlation / against" = scatter; "trend / over time / each month" = line; "compare / total per / for each" = bar; "distribution / frequency" = histogram; "spread / outliers / median per group" = box.

- The `xlabel=` / `ylabel=` keyword arguments of `df.plot` exist in current pandas; if an old pandas rejects them, call `plt.xlabel(..)` afterwards.
- `plt.hist` ignores NaN silently, `plt.plot` leaves a gap - a chart that looks fine can still sit on blanks; clean first.

5.1 Traps for sections 3.11-3.13

Mistake	What you see	Fix
`plt.boxplot(data, labels=[..])`	Matplotlib 3.11: `TypeError: boxplot() got an unexpected keyword argument 'labels'`	`tick_labels=` (3.9+) or `plt.xticks([1,2,3], [names])`
`plt.boxplot(df['A'], df['B'])`	second argument is read as `notch`: `ValueError: truth value of a Series is ambiguous`	ONE list of datasets: `plt.boxplot([df['A'], df['B']])`
`plt.boxplot(df)` on a table with text columns	`TypeError`/garbage	pick numeric columns: `df[['A','B']]`
box plot of data containing NaN	that box silently disappears (median is NaN)	`df['A'].dropna()` first
`df.boxplot(column='Mark', by='Class')` extra grey title "Boxplot grouped by Class"	untidy	`plt.suptitle('')` then your own `plt.title`
xticks for boxes start at 0	labels shifted by one	boxes sit at 1, 2, 3
`vert=False`	deprecated warning on Matplotlib 3.11	`orientation='horizontal'`
scatter with different-length lists	`ValueError: x and y must be the same size`	check `len(x) == len(y)`
scatter with the points joined by a line wanted	`scatter` draws no lines	use `plt.plot(x, y, marker='o')`
line chart looks zig-zag	data not in time order	sort by x before `plt.plot`
`plt.legend()` shows nothing; warning "No artists with labels found"	forgot `label=` in the plot call	`plt.plot(..., label='A')` then `plt.legend()`
bar `color=['red','blue']` with 5 bars	colours repeat	one colour per bar
using a scatter to show "total per product"	meaningless dots	categories -> bar (table 3.13)

6. Anticipated exam questions

Q1 (2) Create a bar chart showing the total quantity sold for each product. Template B (groupby, `plt.bar`, title, both labels).

Q2 (2) Line plot of total sales per month of 2023; x = months, y = total sales. Template C (filter year, groupby `dt.month`, `plt.plot`).

Q3 (2) Show both charts in the same program without them overlapping. Call `plt.figure()` before each chart and `plt.show()` once at the end (template F), or use two separate `fig, ax = plt.subplots()` calls.

Q4 (3) Plot the training loss recorded for each epoch; the x-axis is the epoch number.

```
plt.figure()
plt.plot(range(1, len(losses) + 1), losses, marker='o')
plt.title('Training Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.show()
```

Q5 (3) (Practice Task 3) Histogram of heights with the given title and axis labels. Template L; mention `bins=` changes the bucket width.

Q6 (2) Which chart suits "the share of total sales per product" and why? A pie (parts of a whole) - but a bar chart is easier to read exact values from. Pie code: template E. Say both.

Q7 (3) Draw the bar chart and the monthly line chart side by side in one window. Template G (`plt.subplots(1, 2)`; `ax[0]`, `ax[1]`; `set_title`).

Q8 (2) Save the chart as `sales.png` as well as showing it. `plt.savefig('sales.png')` immediately before `plt.show()`.

Q9 (3) Show total sales per month for 2023 AND 2024 as two lines on one chart with a legend.

```
for year in [2023, 2024]:
    d = df[df['Date'].dt.year == year]
    m = d.groupby(d['Date'].dt.month)['Total'].sum()
    plt.plot(m.index, m.values, marker='o', label=str(year))
plt.title('Monthly Sales by Year')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.legend()
plt.show()
```

Q10 (2) Why did you use `marker='o'`? With few points (only 4 months) the dots show where the data really is; the line only joins them.

6.1 Extra questions on the instructor's checklist

Q11 (2) Draw a scatter plot of hours studied against exam mark with a title and both axis labels. Template 3.11 (scatter). `plt.scatter(hours, marks)`; title; `xlabel('Hours studied')`; `ylabel('Exam mark')`.

Q12 (3) Draw box plots comparing the marks of three classes. Label the boxes and axes. Template 3.12:

```
plt.figure()
plt.boxplot([class_a, class_b, class_c])
plt.xticks([1, 2, 3], ['Class A', 'Class B', 'Class C'])
plt.title('Exam Marks by Class')
plt.xlabel('Class')
plt.ylabel('Mark')
plt.show()
```

Q13 (3) What does the box plot show and which class has the most consistent marks? Median line, quartile box, whiskers, outlier dots. Class C: the shortest box (smallest IQR) = most consistent; Class A has an outlier (95).

Q14 (2) A box plot of heights shows one dot far above the whisker. What is it? An outlier: a value beyond Q3 + 1.5 x IQR. (Check the data: a typo or a genuinely tall student?)

Q15 (2) Which chart type would you use for (a) monthly trend, (b) relationship between two numbers, (c) spread per group (d) totals per category? Say why. line; scatter; box plot; bar (table 3.13).

Q16 (3) Draw a line chart of `months` and `sales` with a dashed line, circle markers, a legend and a grid. Template 3.11 (line).

Q17 (3) Make a box plot of `Mark` for each `Class` straight from the DataFrame `marks`. `marks.boxplot(column='Mark', by='Class')`, then `plt.suptitle('')`, title, labels, `plt.show()`.

7. Quick-reference box (hand-copy this)

```
IMPORT   import matplotlib.pyplot as plt
NEW PAGE plt.figure(figsize=(8,5))    <- one per chart, or charts overlap!
BAR      s = df.groupby('P')['Q'].sum()      plt.bar(s.index, s.values)
LINE     plt.plot(x, y, marker='o')          label='A' + plt.legend()
HIST     plt.hist(values, bins=5, edgecolor='black')
SCATTER  plt.scatter(x, y)
PIE      plt.pie(s, labels=s.index, autopct='%1.1f%%')
CAPTIONS plt.title('..')  plt.xlabel('..')  plt.ylabel('..')
TIDY     plt.xticks(rotation=45)    plt.tight_layout()    plt.grid(True)
OUTPUT   plt.savefig('a.png')  THEN  plt.show()      (savefig BEFORE show)
SUBPLOT  fig, ax = plt.subplots(1, 2, figsize=(10,4))
         ax[0].bar(..)  ax[0].set_title('..')  ax[0].set_xlabel('..')
         fig.tight_layout()
PANDAS   s.plot(kind='bar'|'line'|'hist'|'pie', title='..')  then plt.show()
MONTHS   d = df[df['Date'].dt.year==2023]
         m = d.groupby(d['Date'].dt.month)['Total'].sum()   (x = 1..12)
LOSS     plt.plot(range(1, len(losses)+1), losses)  + title/Epoch/Loss labels
HEIGHTS  title 'Height Distribution'  x 'Height (cm)'  y 'Frequency'
TRAPS    plot(PeriodIndex) fails -> .astype(str) | groupby BEFORE bar
--- instructor checklist additions ---
SCATTER  plt.scatter(x, y, color=, s=size, alpha=, marker='o')  2 numbers/item
LINE     plt.plot(x, y, marker='o', linestyle='--', linewidth=2, label='A')
BAR      plt.bar(names, heights, color=, width=0.6, edgecolor='black')
BOX      plt.boxplot([g1, g2, g3])   plt.xticks([1,2,3], ['A','B','C'])
         NOT labels=  (gone in Matplotlib 3.11); tick_labels= = 3.9+ only
         df.boxplot(column='V', by='G'); plt.suptitle('')
         df.plot(kind='box')    boxes sit at 1,2,3...
READ BOX line=median, box=Q1..Q3 (IQR), whiskers=1.5*IQR, dots=outliers
WHICH    trend->line  relation->scatter  compare->bar  spread->hist
         spread/outliers per group->box   parts of whole->pie
EVERY plot: plt.title('..')  plt.xlabel('..')  plt.ylabel('..')
```